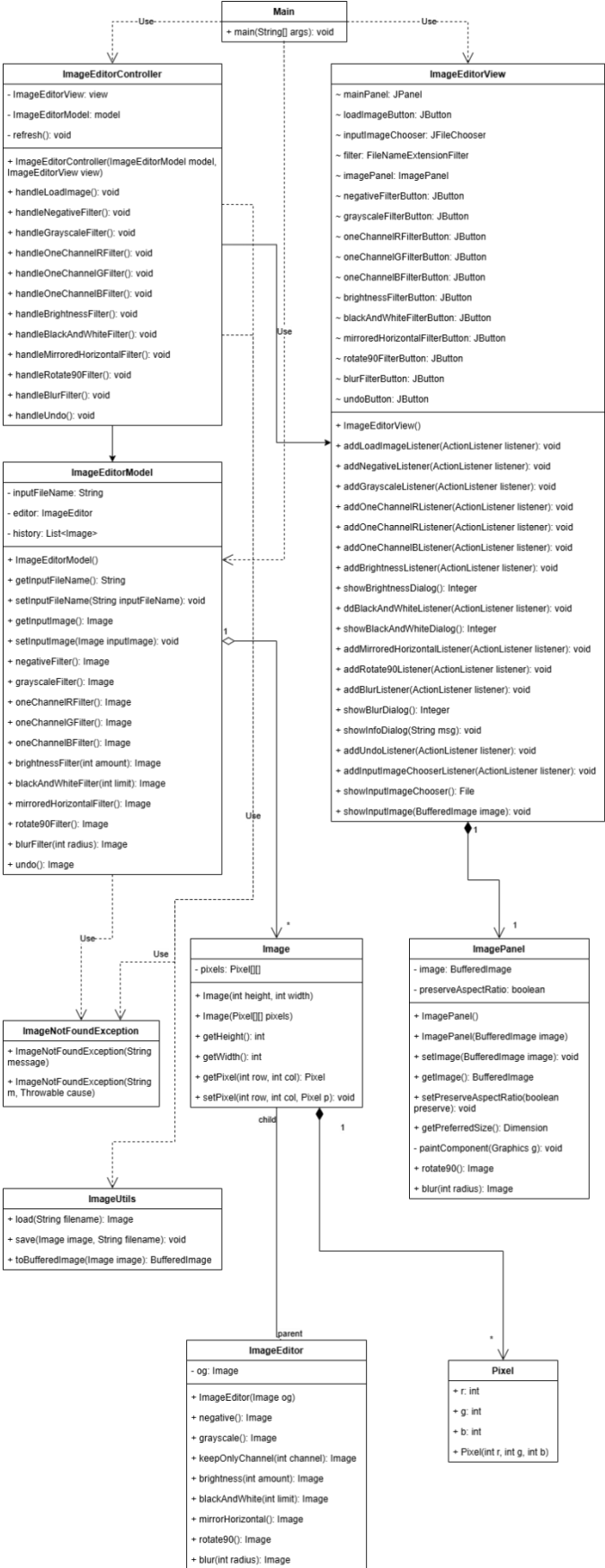


Ejercicio 3 - Editor de Filtros de Imágenes

Análisis:

1. Interfaz gráfica de usuario con estilo personalizado: El sistema debe tener una GUI hecha con Swing y usar FlatLaf para su diseño visual.
2. Selección y carga de imágenes: El usuario debe poder abrir imágenes locales en formato PNG usando un selector de archivos con filtro de extensión.
3. Representación matriz de píxeles: El sistema debe convertir la imagen cargada en una matriz de objetos Pixel con canales RGB para poder modificarla.
4. Aplicación del filtro negativo: El sistema debe permitir invertir los colores RGB de la imagen mediante un botón.
5. Aplicación del filtro escala de grises: El sistema debe promediar los canales RGB de cada píxel y asignar el mismo valor a los tres canales al presionar el botón correspondiente.
6. Filtros de aislamiento de canales: El sistema debe tener tres botones para conservar solo un canal de color (Rojo, Verde o Azul) y poner los otros dos en 0.
7. Ajuste de brillo con diálogo dinámico: El sistema debe permitir cambiar el brillo con un JOptionPane, aceptando valores entre -255 y 255 y manteniendo los colores entre 0 y 255.
8. Conversión a blanco y negro: El sistema debe pedir un umbral entre 0 y 255 mediante un JOptionPane y convertir los píxeles en blanco (255,255,255) o negro (0,0,0).
9. Espejado horizontal de la imagen: El sistema debe permitir reorganizar las columnas de píxeles para reflejar la imagen horizontalmente.
10. Rotación de 90 grados: El sistema debe permitir rotar la imagen 90° a la derecha y ajustar el ancho y alto de la matriz.
11. Aplicación de desenfoque: El sistema debe permitir aplicar un filtro de suavizado con un radio de desenfoque entre 0 y 10.
12. Registro en el historial: Cada transformación realizada debe guardarse en orden y sin límite en una lista dinámica.
13. Deshacer última operación: El sistema debe permitir deshacer la última modificación, eliminando el último estado de la lista y restaurando la imagen anterior.
14. Visualización continua de la imagen resultante: El sistema debe actualizar el ImagePanel inmediatamente después de cargar o modificar la imagen.
15. Control y mensajería de excepciones: El sistema debe mostrar una excepción ImageNotFoundException al intentar filtrar o deshacer sin una imagen cargada. El Controlador debe manejarla y mostrar un mensaje al usuario sin cerrar la aplicación.

Diseño:



Preguntas:

1. ¿Por qué se utiliza una colección dinámica y no un arreglo de tamaño fijo para almacenar el historial de la aplicación?

Se utiliza una colección dinámica porque la cantidad de filtros que aplicará el usuario puede variar. A diferencia de un arreglo fijo, `ArrayList` puede crecer según sea necesario, evitando limitar el historial o desperdiciar memoria.

2. ¿Qué diferencia existe entre `List` y `ArrayList`? ¿Por qué se recomienda declarar la variable utilizando el tipo `List`?

`List` es una interfaz que define las operaciones de una lista, mientras que `ArrayList` es una clase que la implementa usando un arreglo redimensionable. Se declara como `List` porque permite cambiar la implementación, por ejemplo a `LinkedList`, sin tener que modificar el resto del código.

3. ¿Qué operaciones de la colección dinámica utilizó para agregar, recorrer y eliminar elementos del historial?

En la clase `ImageEditorModel` se utilizan diferentes métodos para manejar el historial de imágenes: `add(element)` para agregar nuevas versiones, `removeLast()` para eliminar la última modificación al usar `undo()`, e `isEmpty()`, `size()`, `getFirst()` y `getLast()` para verificar, contar y acceder a las imágenes almacenadas.

4. ¿Qué situaciones dentro del programa pueden producir una excepción?

Dentro de la aplicación pueden ocurrir varios errores: intentar aplicar un filtro o deshacer sin una imagen cargada, errores al cargar o guardar archivos (`IOException`), seleccionar formatos no válidos (`NullPointerException`) e ingresar valores no numéricos en Brillo, Umbral o Desenfoque (`NumberFormatException`).

5. ¿Cuáles excepciones propias creó y que error representa cada una?

Se creó la excepción personalizada `ImageNotFoundException`, que extiende de `Exception` y se utiliza cuando se intenta aplicar un filtro sin tener una imagen cargada. También se recomienda crear una segunda excepción, como `EmptyHistoryException`, para manejar el deshacer de operaciones.

6. ¿En qué parte del programa se lanzan las excepciones y en qué parte se capturan? ¿Por qué es conveniente esa separación?

Las excepciones se lanzan desde el Modelo (`ImageEditorModel`), donde se encuentra la lógica de la aplicación, y se capturan en el Controlador (`ImageEditorController`), que muestra el error al usuario mediante la Vista (`ImageEditorView`). Esta separación mantiene el patrón MVC y permite que el Modelo funcione sin depender de componentes visuales.

7. ¿Qué acción realiza mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación?

El bloque finally se utiliza para liberar recursos, como el contexto gráfico Graphics2D, mediante g2.dispose(). Se ejecuta siempre, incluso si ocurre un error, evitando fugas de memoria y asegurando que los recursos del sistema se liberen correctamente.

8. ¿Qué responsabilidad tiene cada uno de los tres componentes del patrón MVC dentro de su programa?

El Modelo (ImageEditorModel, ImageEditor, Image y Pixel) maneja los datos, el historial y los filtros de la imagen. La Vista (ImageEditorView e ImagePanel) se encarga de la interfaz gráfica y de mostrar la imagen, botones y diálogos. El Controlador (ImageEditorController) conecta ambos, recibe las acciones del usuario, ejecuta las operaciones del Modelo, maneja los errores y actualiza la Vista.

9. Por qué las clases del modelo no deben importar en javax.swing? ¿Qué ventaja ofrece esta separación?

Las clases del Modelo no deben depender de Swing porque solo manejan la lógica y los datos de las imágenes. Esto permite reutilizar el código en otras aplicaciones, cambiar la interfaz sin afectar los filtros y facilita realizar pruebas unitarias con JUnit sin necesidad de usar componentes gráficos.